



VESTORA

ملف مذاكرة · مادة تحضير شخصية للمناقشة

Testing

اختبارات التكامل الثمانية والعشرون، الأمان، وخطة الاختبار اليدوي — أربعة وستون سيناريو

هذا الملف للمذاكرة فقط. ليس تقريرًا ولا وثيقة تُسلّم — الغرض منه أن تفهم جزءك من المشروع فهمًا كاملاً وتقدر تشرحه وتدافع عنه.

المحتويات

1. Your Responsibility, Exactly	3
1.1. The One-Line Answer	3
1.2. What You Own, in Detail	3
1.3. What's Not Yours	3
2. Testing Types — What Exists and Why	4
2.1. The Four Types in This Project	4
2.2. The Test Pyramid	5
3. Integration Test Architecture	5
3.1. The Basic Idea	5
3.2. Test Isolation and Test Data	5
4. Auth Tests — Nine Facts, Proven Live	6
5. Projects Tests — Ownership Is Not a Role Check	7
6. Payments Tests — Proving the Money Rule	8
6.1. The Full Journey, One Test	8
6.2. The Rest of the Suite	9
7. Security Tests — Thirteen Real Attacks, All Failed	9
7.1. The Seven Tests	10
8. The Manual Test Plan — Sixty-Five Scenarios, All Pass	11
8.1. 1. 11	المصادقة والحسابات
8.2. 2. 12	الأونبوردنج
8.3. 3. 12	لوحة المؤسّس
8.4. 4. 12	لوحة المستثمر
8.5. 5. 13	البحث المحفوظ وقائمة المتابعة
8.6. 6. 13	التصفّح والاكتشاف
8.7. 7. 13	صفحة المشروع الواحد
8.8. 8. 13	غرفة الصفقة
8.9. 9. 14	الرسائل والإشعارات
8.10. 10. 14	الإعدادات والملف الشخصي
8.11. 11. 14	الأدمن
8.12. 12. 14	الدفع
8.13. 13. 15	الصفحات الثابتة وصفحات الحالة
8.14. 14. الاتجاه العربي (RTL)	15
8.15. 15. الاستجابة (Responsive)	15
9. Test Coverage	16
9.1. Tests Count Is Not Coverage Percentage	16
10. Regression Testing	16
11. Testing Limitations / Out of Scope	16
12. Terms You Must Know	17
13. Doctor Questions and Pressure Follow-ups	17
14. Suggested Study Plan	20
15. Final Code Map — Where to Find Everything	20
16. Cheat Sheet — Review Before You Walk In	21

1. Your Responsibility, Exactly

1.1. The One-Line Answer

إنت مسؤول عن إثبات إن الباكد شغال زي ما بيدّعي: 28 اختبار تكامل بتنده الـ API الحقيقي، توثيق منهجي لمحاولات اختراق حقيقية، وخطة اختبار يدوي كاملة للفرونّت بأربعة وستين سيناريو.

اعرف ده كويس

الجملة اللي تقولها لو اتسألت «إنت عملت إيه في التستينج؟»:

«أنا بنيت طبقة اختبارات تكامل فوق الـ 48 اختبار الأصلية اللي كانت مقتصرة على المنطق الرياضي بس — بتنده الـ API الحقيقي على قاعدة بيانات معزولة، وبتغطّي المصادقة والمشاريع والدفع كاملين. وثّقت 13 محاولة اختراق حقيقية اتنفّذت وفشلت كلها. ونفّذت خطة اختبار يدوي منظّمة على الفرونّت كله — 65 سيناريو، النتيجة النهائية All Pass.»

1.2. What You Own, in Detail

المجال	العدد	الملفات
اختبارات التكامل	28	AuthEndpointsTests.cs ProjectsEndpointsTests.cs SecurityBoundaryTests.cs PaymentsEndpointsTests.cs
بنية الاختبار	3	CustomWebApplicationFactory.cs TestHelpers.cs TestLogCapture.cs
توثيق الأمان	13 محاولة	SECURITY-FINDINGS.md
الاختبار اليدوي	65 سيناريو	FRONTEND-MANUAL-TEST-PLAN.md

جدول 1 نصيبك: 76 اختبار آلي (28 جديد + 48 قديم) + 65 سيناريو يدوي.

اعرف ده كويس

الرقم اللي تحفظه: 76 اختبار آلي، كلهم ناجحين، صفر فشل. والاختبار اليدوي: 65 سيناريو، النتيجة النهائية All Pass.

1.3. What's Not Yours

مش جزءك

- الـ 48 اختبار الأصلية على FundingMath و PipelineStages — دي موجودة قبل ما تبدأ، ومكتوبة على المنطق الرياضي البحث. إنت ضفت طبقة فوقها، ماكتبتهاش.
- كود الباكد نفسه (الكنترولز والخدمات) — بتاع عضو الباكد وقائد الفريق. إنت بتختبره، مش بتكتبه.
- كود الفرونّت نفسه — بتاع عضو الفرونّت. إنت بتجربّه بإيدك، مش بتعدّل فيه.
- أتمتة المتصفّح (E2E بـ Playwright) — اتقيّمت وتقرّر إنها مش هتعمل. النطاق اتّفق عليه: تكامل آلي للباكد + اختبار يدوي منظّم للفرونّت، و خلاص.
- لو اتسألت في أي حاجة من دول، سلّمها لصاحبها بوضوح.

اختبر نفسك

1. إيه دورك في التستينج بالطبط؟
28 اختبار تكامل على الباكند الحقيقي، توثيق 13 محاولة اختراق، وخطة اختبار يدوي بـ 65 سيناريو للفرونت.
2. الـ 48 اختبار القديمة بتاعتك؟
لأ — دي كانت موجودة قبلي، على المنطق الرياضي بس. أنا ضفت طبقة اختبارات تكامل فوقها.
3. ليه مفيش Playwright؟
اتقيّم النطاق واتقرر إن التكامل الآلي + الاختبار اليدوي المنظم كافيين لحجم المشروع ده.

2. Testing Types — What Exists and Why

قبل أي تفصيلة تقنية، لازم تعرف تفرّق بين أنواع الاختبار الأربعة اللي في المشروع — ودي أساس أي سؤال هيتسأل عن التستينج.

2.1. The Four Types in This Project

النوع	مين كتبه	بيثبت إيه
Unit	موجود قبلك — 48 اختبار	دالة واحدة لوحدها، من غير قاعدة بيانات ولا سيرفر. «لو مشروع محتاج 100 ألف وجمع 40 ألف، FundingMath يرجع 40%؟»
Integration	إنت — 28 اختبار	API الحقيقي شغال، بيتنده فعليًا، وبيحفظ ويقرا من قاعدة بيانات (معزولة). «لو بعت POST تسجيل فعليًا، هل المستخدم اتسجل صح؟»
Security	إنت — جزء من الـ 28	محاولة اختراق حقيقية عمداً. «مؤسس يقدر يعدّل مشروع مؤسس ثاني؟»
Manual	إنت — 65 سيناريو	إنسان بيفتح المنتج بإيده ويجرب، ويسجل النتيجة. مفيش أداة، مفيش كود — عين بشرية.

جدول 2 أنواع الاختبار الأربعة، ومين كتب كل واحد.

اعرف ده كويس

ليه محتاجين النوعين الأول والثاني مع بعض؟

Unit سريع جدًا ودقيق، بس بيثبت إن الدالة لوحدها صح — مش إن API كله بيشتغل مع بعض. Integration أبطأ، بس بيثبت إن الطبقات كلها (routing، model binding، قاعدة البيانات، authorization) شغالة مع بعض.

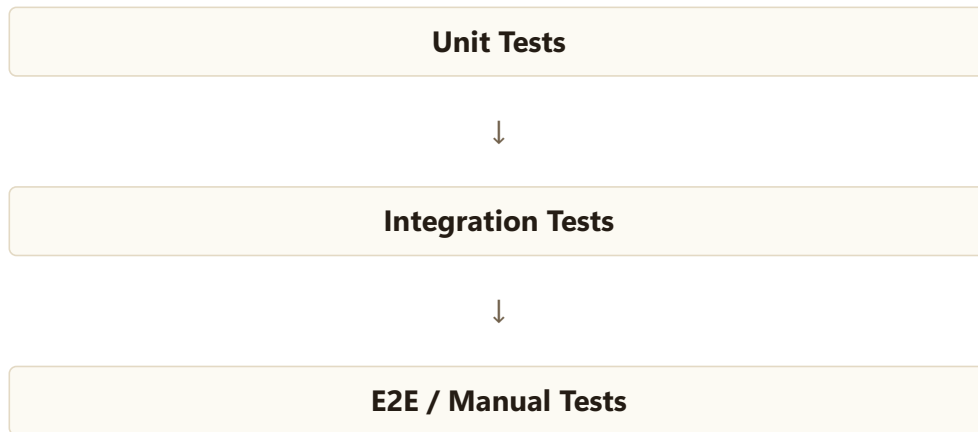
التعليق في AGENTS.md يقول صراحة: «الاختبارات بتغطّي طبقة الدومين بس... مفيش integration ولا authorisation ولا payment tests». — يعني السويت بتاعتك سدّت فجوة موثقة ومعروفة، مش حاجة مخترعة من عندك.

خد بالك

ليه مفيش Performance/Load Testing؟

ده سؤال محتمل. الإجابة الأمينة: المشروع مشروع تخرج، مش سيرفر إنتاج بيستحمل آلاف المستخدمين. اختبار حمل هنا هيبقى اختبار وهمي — بيقيس رقم مالوش معنى حقيقي في السياق ده. القرار الواعي أهم من التغطية الشكلية.

2.2. The Test Pyramid



اعرف ده كويس

كل طبقة موجودة لسبب مختلف: **Unit** بيتأكد إن الحساب نفسه صح (`FundingMath`). **Integration** بيتأكد إن الطبقات شغالة مع بعض (auth، routing، قاعدة بيانات). **Manual** بيتأكد إن اللي المستخدم فعلياً شايفه في المتصفح مطابق للي الكود بيقله. كل طبقة بتغطّي حاجة الطبقة اللي قبلها مبتقدرش تشوفها.

3. Integration Test Architecture

الفصل ده بيشرح `CustomWebApplicationFactory.cs` — أهم حتّة بنيتها، وهي اللي بتشغّل API الحقيقي على قاعدة بيانات معزولة لكل اختبار.

3.1. The Basic Idea

اعرف ده كويس

`WebApplicationFactory<Program>` بتشغّل API **كامل** — كل الكنترولرز، كل الخدمات، كل middleware — لكن **في الذاكرة**، من غير ما تفتح بورت حقيقي. الاختبار بيعت طلب HTTP حقيقي، والAPI بيردّ زي ما هيردّ في الإنتاج بالضبط، ما عدا حاجة واحدة: قاعدة البيانات.

3.2. Test Isolation and Test Data

اعرف ده كويس

كل اختبار بيشتغل على **قاعدة بيانات معزولة في الذاكرة** (EF Core InMemory) — مش الداتابيز الحقيقية المشتركة بتاعة الفريق. القاعدة دي بتتعمل من جديد لكل كلاس اختبار، فمفيش اختبار ممكن يآثر على بيانات الفريق الحقيقية بالغلط.

اعرف ده كويس

TestHelpers.cs هو المسؤول عن تجهيز البيانات: بيسجل حساب حقيقي (`RegisterUserAsync`)، بيسجل دخول ويرجع client مسجل (`LoginAsClientAsync`)، ويبذر مشروع أو استثمار معتمد مباشرة في قاعدة البيانات (`SeedApprovedProjectAsync` , `SeedApprovedInvestmentAsync`) لما اختبار محتاج حالة جاهزة من غير ما يمر بكل خطوة يدويًا.

خد بالك

كل اختبار لازم يجهز البيانات اللي هو محتاجها بنفسه، ومايعتمدش على إن اختبار ثاني اتنقذ قبله أو رتب حاجة معينة. عشان كده كل اختبار في السويت يبدأ بتسجيل مستخدم أو بذر بيانات جديدة بإيميل عشوائي (`Guid.NewGuid()`) — مش يفترض حساب أو مشروع موجود من اختبار سابق.

فين ده في الكود

`Integration/CustomWebApplicationFactory.cs`
`Integration/TestHelpers.cs`
`Integration/TestLogCapture.cs`

بنية تشغيل الاختبارات — قاعدة بيانات معزولة
 التسجيل، الدخول، البذر المباشر للداتايز
 أداة تصحيح — بتلقط الأخطاء اللي الـ exception handler بيخفيها

اختبر نفسك

1. الاختبارات بتلمس الداتايز الحقيقية؟
 لا — كل كلاس اختبار بيشتغل على قاعدة بيانات InMemory معزولة، بتعمل من جديد كل مرة.
2. لو اختبار محتاج مستخدم أو مشروع جاهز، بيحبيه مين؟
 من TestHelpers — بيسجل حساب حقيقي عن طريق الـ API، أو يبذر بيانات مباشرة في الداتايز للحالات اللي مش محتاجة تمر بكل خطوة.
3. الاختبارات بتعتمد على ترتيب معين؟
 لا — كل اختبار بيجز بياناته بنفسه بإيميل عشوائي، مش يفترض حاجة من اختبار سبقه.

4. Auth Tests — Nine Facts, Proven Live

AuthEndpointsTests.cs بيثبت تسع حقائق عن **AuthService**، كل واحدة بطلب HTTP حقيقي مش بقراءة كود.

الاختبار	بيثبت إيه
<code>Register_WithValidData_CreatesAccountAndReturns201</code>	الحساب بيتعمل فعليًا، وكلمة السر متخزنة مباشرة مش نص واضح.
<code>Register_WithDuplicateEmail_Returns400</code>	مفيش حسابين بنفس الإيميل.
<code>Register_WithWeakPassword_Returns400WithFieldLevelError</code>	الرفض على مستوى الحقل ، مش رسالة عامة.
<code>Login_WithCorrectCredentials_ReturnsTokenPair</code>	الدخول بيرجع access token و refresh token فعليين.
<code>Login_UnknownEmailAndWrongPassword_ReturnIdenticalMessages</code>	رسالة الفشل متطابقة بالحرف بين إيميل مش موجود وكلمة سر غلط — منع تسريب المستخدمين.

الاختبار	بيثت إيه
Login_AfterFiveFailedAttempts_LocksTheAccount	بعد 5 محاولات فاشلة، المحاولة السادسة بترفض حتى لو كانت هتنجح.
RefreshToken_Rotates_OldTokenNoLongerWorks	كل استعمال لتوكن التحديث بيولّد واحد جديد، والقديم بيطل.
RefreshToken_ReuseOfRevokedToken_RevokesAllActiveSessions	استخدام توكن سبق تدويره بيعتبر سرقة، وكل جلسات الحساب بتتلغي.
VerifyEmail_WithWrongCode_Returns400	كود تحقق غلط مبيأكدش الحساب.

جدول 3 تسع حقائق عن المصادقة، كل واحدة اثبتت بطلب HTTP حقيقي.

اعرف ده كويس

أقوى اختبار في الملف كله: RefreshToken_ReuseOfRevokedToken_RevokesAllActiveSessions

السيناريو اللي بيثبتته: مستخدم دوّر توكنه (استعمله وترقى لتوكن جديد). بعدين حد استخدم التوكن **القديم** (سيناريو سرقة). الاختبار بيتأكد إن **كل** التوكنات النشطة بتاعة الحساب اتلغت — مش بس المحاولة اللي فشلت.

```
1 using var verifyDb = Db();
2 var tokens = await verifyDb.RefreshTokens.Where(t => t.UserId ==
userId).ToListAsync();
3 Assert.All(tokens, t => Assert.NotNull(t.RevokedAtUtc));
```

لو الدكتور سأل «إيه أصعب اختبار كتبتة؟» — ده هو، لأنه بيثبت سلوك أمني معقد مش مجرد «الرد كان 200».

خد بالك

اختبار القفل (Login_AfterFiveFailedAttempts_LocksTheAccount) بيعتمد على رقم حقيقي من appsettings.json: MaxFailedLoginAttempts = 5. لو الرقم ده اتغير يومًا، الاختبار هيفشل — وده **مقصود**، عشان يفضل الاختبار متزامن مع الإعداد الحقيقي مش رقم مكتوب بالغلط في مكانين.

5. Projects Tests — Ownership Is Not a Role Check

بيثت 6 حقائق، أهمها كلها عن الفرق بين الدور والملكية. ProjectsEndpointsTests.cs

الاختبار	بيثت إيه
CreateProject_AsInnovator_StartsHiddenPendingReview	مشروع جديد بيتخبي لحد المراجعة — مش ظاهر فورًا.
CreateProject_AsInvestor_IsForbidden	مستثمر مايقدرش ينشئ مشروع خالص.
Facets_CountEachSectorIndependentlyOfItsOwnFilter	عدّاد قطاع معيّن بيفضل صحيح حتى وهو نفسه مطبّق كفلتر.
CloseRound_WithdrawsOpenFundingRequest_ButNotAPaidOne	إقفال جولة بيسحب طلب تمويل مفتوح، بس مش بيلمس اللي اتدفع.

الاختبار	بيثت إيه
<code>UpdateProject_ByNonOwnerInnovator_IsForbidden</code>	مؤسس ثاني (نفس الدور بالظبط) مايقدرش يعدّل مشروع مش بتاعه.
<code>CloseRound_ByNonOwner_IsForbidden</code>	نفس المبدأ، على إقفال الجولة.

جدول 4 ست حقائق عن المشاريع، اتنين منهم عن الفرق بين الدور والملكية.

اعرف ده كويس

الحقيقة اللي أغلب الناس بتنساها: `[Authorize(Roles = "Innovator")]` بتأكد إنك مؤسس — مش إنك صاحب المشروع ده. مؤسس ثاني دوره برضه `Innovator`، وبرضه هيتلغى بـ 403 لو حاول يعدّل مشروع مش بتاعه.

```
1 var otherFounder = await TestHelpers.LoginAsClientAsync(_factory, otherEmail,
otherPassword);
2 var response = await otherFounder.PutAsJsonAsync($"/api/projects/{project.Id}",
new {...});
3 Assert.Equal(HttpStatusCode.Forbidden, response.StatusCode);
```

ده الاختبار اللي بيثبت قاعدة رقم 4 في `AGENTS.md` بالكود، مش بالكلام.

6. Payments Tests — Proving the Money Rule

بيثبت 6 حقائق، وواحد منهم بيمشي في رحلة الدفع كاملة، من الطلب لحد التسوية. `PaymentsEndpointsTests.cs`

6.1. The Full Journey, One Test

بتعمل أربع خطوات حقيقية `FullPaymentJourney_RecordsEventBeforeMarkingTransactionSucceeded` بالترتيب:

الخطوة	الـ endpoint
1. المؤسس بيطلب فلوس	<code>POST investments/{id}/funding-request</code>
2. المستثمر يفتح checkout	<code>POST funding-requests/{id}/checkout</code> — المزود الوهمي بيرجع session
3. المستثمر «يدفع» (محاكاة)	<code>POST sandbox/sessions/{id}/resolve</code> بنتيجة <code>success</code>
4. تأكيد النتيجة	<code>POST transactions/{id}/verify</code> — دي الخطوة اللي بتطبق النتيجة فعليًا

جدول 5 رحلة الدفع الكاملة، أربع خطوات حقيقية.

اعرف ده كويس

كل الرحلة دي على مزود دفع وهمي (`SimulatedPaymentProvider`) — مفيش مفتاح Stripe حقيقي ولا اتصال إنترنت. `Payments:Provider` في `appsettings.json` قيمتها الافتراضية `"simulated"` أصلاً، فمفيش أي إعداد إضافي محتجته.

التعليق في كود المزود الوهمي يقول: «بيفتح جلسة، بيسيبها معلقة لحد ما حد يتصرف فيها، ويرجع بالطبط اللي أي مزود حقيقي هيرجعه — الحاجة الوحيدة اللي مش بيعملها إنه بيكلّم الإنترنت.» يعني الاختبار بيمشي على نفس الآلة الحقيقية (state machine)، مش نسخة مبسطة.

6.2. The Rest of the Suite

الاختبار	بيثبت إيه
<code>FailedPayment_LeavesFundingRequestOpenForRetry</code>	دفعة فاشلة ما بتغفلش الطلب — المستثمر يقدر يعيد المحاولة.
<code>CreateFundingRequest_AsInvestor_IsForbidden</code>	بس المؤسس يقدر يطلب فلوس، مش المستثمر.
<code>Checkout_AsFounder_IsForbidden</code>	المؤسس اللي طلب الفلوس ما يقدرش يكون هو اللي بيدفعها.
<code>Checkout_ByUninvolvedInvestor_IsForbidden</code>	مستثمر تالت مالوش علاقة بالطلب ما يقدرش يدفعه.
<code>Checkout_OnAlreadyPaidRequest_IsRejected</code>	طلب اتدفع قبل كده ما يقدرش يتدفع ثاني — منع الدفع المزدوج.

جدول 6 خمس حقايق تانية عن حدود الدفع.

فين ده في الكود

`Integration/PaymentsEndpointsTests.cs`
`Integration/PaymentsEndpointsTests.cs · FullPaymentJourney...`

الرحلة الكاملة + 5 حدود صلاحية
 أهم اختبار في الملف

اختبر نفسك

1. رحلة الدفع الكاملة بتكوّن من كام خطوة؟
 أربعة: طلب تمويل، فتح checkout، محاكاة الدفع، ثم verify — والverify هي اللي بتطبّق النتيجة فعلياً.
2. ليه استخدمت المزود الوهمي مش Stripe حقيقي؟
`Payments:Provider` الافتراضية «simulated»، وهي بتمشي على نفس آلة الحالة الحقيقية من غير اتصال إنترنت.
3. إيه اللي بيمنع الدفع المزدوج؟
 فحص صريح في `PaymentService`: طلب `Status == Paid` يرفض أي محاولة checkout جديدة عليه.

7. Security Tests — Thirteen Real Attacks, All Failed

واحدة انتفّذت كطلب HTTP فعلي، مش قراءة كود والثقة فيه. `SecurityBoundaryTests.cs` (7 اختبارات) و `SECURITY-FINDINGS.md` بيوثّقوا 13 محاولة اختراق حقيقية. كل

7.1. The Seven Tests

الاختبار	المحاولة
AdminOnlyEndpoint_RejectsAnAuthenticatedNonAdmin	مستثمر عادي يحاول يوصل لتدقق النشاط (أدمن بس).
AdminOnlyEndpoint_AcceptsARealAdmin	التأكيد العكسي: أدمن حقيقي يقدر يوصل — الحماية مش بتمنع الكل.
ProtectedEndpoint_WithNoToken_Returns401	header Authorization طلب من غير أي خالص.
ProtectedEndpoint_WithTamperedToken_Returns401	توكن حقيقي اتغير فيه حرف واحد في التوقيع.
InvestorOnlyEndpoint_RejectsAnInnovator	مؤسس يحاول يكتب تزكية على مشروعه هو (مقصورة على المستثمرين).
PaymentTransaction_IsInvisibleToAnUninvolvedThirdParty	مستثمر ثالث يحاول يشوف تفاصيل دفع مش طرف فيه.
InvestorDirectory_HidesInvestorsWhoDidNotOptIn	مستثمر عطل ظهوره في الدليل — هل بيظهر رغم كده؟

جدول 7 سبع محاولات اختراق مباشرة، كل واحدة برد HTTP حقيقي.

اعرف ده كويس

تفصيلة تقنية دقيقة في اختبار التوكن المتلاعب فيه:

أول محاولة كانت بتغير آخر حرف في التوكن، وفشلت — لأن base64url فيها بتات padding زيادة، وأحياناً آخر حرف بيتغير من غير ما يغير البايتات الفعلية. الحل: غيرت حرف في نص جزء التوقيع، مش آخره.

```
1 var midSignature = signatureStart + (realToken.Length - signatureStart) / 2;
```

درس عملي: حتى وقت تعمل هجوم وهمي، لازم تفهم شكل البيانات اللي بتلعب فيها.

خد بالك

الاختبار الأخير (`InvestorDirectory_HidesInvestorsWhoDidNotOptIn`) مش ثغرة أمنية بالمعنى الدقيق — البيانات مش حساسة قد كده — بس هو اختبار **خصوصية** مهم: المستخدم قال «متظهرنيش» والنظام لازم يحترم قراره.

فيين ده في الكود

Integration/SecurityBoundaryTests.cs
SECURITY-FINDINGS.md

سبع محاولات اختراق مباشرة
توثيق الـ 13 محاولة كاملة بالتفصيل

اختبر نفسك

1. كام محاولة اختراق اتوثقت؟

13 محاولة — 7 منهم اختبارات كود مباشرة، والباقي موثّق ضمن اختبارات المصادقة والمشاريع والدفع.

2. ليه فحصت إن أدمن حقيقي بقدر يوصل مش بس إن غيره مايقدرش؟

عشان أثبت إن الحماية بتفرّق فعلاً بين مين مسموح ومين لأ، مش بترفض الكل.

3. التوكن المتلاعب فيه اتغيّر إزاي؟

حرف في نص جزء التوقيع، مش آخره — آخر حرف ممكن يتغيّر من غير ما يغيّر البايتات الفعلية بسبب padding الـbase64url.

8. The Manual Test Plan — Sixty-Five Scenarios, All Pass

خطة الاختبار اليدوي بتغطّي كل صفحة في الفرون — الفرون مفيش فيه أي أداة اختبار آلي أصلاً، فده الإثبات العملي الوحيد إنه شغّال.

اعرف ده كويس

النتيجة النهائية: All Pass على الـ65 سيناريو كلهم. الجدول الكامل تحت — لو الدكتور سأل عن أي صفحة بالتحديد، هتلاقها هنا بالطبط.

المصادقة والحسابات 8.1.1

رقم	السيناريو	المتوقع
1.1	تسجيل حساب جديد	ينقل لصفحة تأكيد الإيميل
1.2	تسجيل بكلمة سر ضعيفة	خطأ جنب حقل كلمة السر بالتحديد
1.3	تسجيل بإيميل مستخدم	خطأ جنب حقل الإيميل
1.4	توفيت التحقق أثناء الكتابة	الخطأ يظهر بعد الخروج من الحقل، مش أول حرف
1.5	تصحيح فوري بعد أول خطأ	الخطأ يختفي فوراً وانت بتكتب
1.6	رفع ملف غير صورة	خطأ جنب مكان الصورة، منفصل عن باقي الأخطاء
1.7	دخول بيانات صحيحة	ينقل للوحة المناسبة للدور
1.8	دخول بكلمة سر غلط	رسالة عامة، من غير تحديد إيه اللي غلط
1.9	نسيت كلمة السر	رسالة إن كود اتبعت
1.10	إعادة تعيين كلمة السر	كلمة السر تتغيّر ويقدر يدخل بيها
1.11	تأكيد الإيميل بكود غلط	رسالة خطأ، من غير تأكيد الحساب
1.12	إعادة إرسال كود التحقق	كود جديد يتبعت

جدول 8 12 سيناريو — المصادقة والحسابات.

8.2. 2. الأونبوردنج

رقم	السيناريو	المتوقع
2.1	ظهور الأونبوردنج أول مرة	شاشة اختيار القطاعات تظهر
2.2	اختيار قطاعات متعددة	كل قطاع مختار يتغير شكله بصرياً
2.3	تخطي الأونبوردنج	يؤدي للوحة مباشرة
2.4	عدم تكرار الأونبوردنج	ما يظهرش ثاني بعد logout/login

جدول 9 4 سيناريوهات — الأونبوردنج.

8.3. 3. لوحة المؤسس

رقم	السيناريو	المتوقع
3.1	حالة فاضية أول دخول	شاشة موجهة بخطوات مرقمة
3.2	إنشاء مشروع جديد	يظهر في «مشاريعي» بحالة قيد المراجعة
3.3	معاينة حية أثناء الإنشاء	الكارت يتحدث فوراً وإن كنت بتكتب
3.4	تعديل مشروع مرفوض	يرجع لطاير المراجعة تلقائياً
3.5	إيقاف/استئناف مشروع	يختفي من التصفح وقت الإيقاف، يرجع بعد الاستئناف
3.6	طلب تمويل	الطلب يظهر للمستثمر بحالة مفتوح
3.7	إقفال جولة تمويل	الجولة تتقفل، وأي طلب معلق يتسحب
3.8	نشر تحديث للمشروع	يظهر في الخط الزمني، والمستخدمين يأخذوا إشعار
3.9	تحليلات المشروع	أرقام المشاهدات ومنحنى الالتزام والتمويل منفصلين

جدول 10 9 سيناريوهات — لوحة المؤسس.

8.4. 4. لوحة المستثمر

رقم	السيناريو	المتوقع
4.1	حالة فاضية أول دخول	حالة موجهة تؤدي لصفحة التصفح
4.2	إبداء اهتمام بمشروع	يتسجل الطلب ويظهر عند المؤسس
4.3	حفظ مشروع	يظهر في المحفوظات والأيقونة تتغير
4.4	متابعة مؤسس	لا يقدر يتابع نفسه، ويقدر يلغي المتابعة
4.5	محفظتي	المشروع يظهر بحالة الالتزام الصحيحة
4.6	كتابة ترقية	يقدر يكتب، تظهر كـ «X من Y» مش نجوم
4.7	ترقية قبل التمويل الكامل	رسالة رفض واضحة
4.8	دليل المستثمرين من عين مؤسس	يظهر المفعّلين للظهور بس

جدول 11 8 سيناريوهات — لوحة المستثمر.

البث المحفوظ وقائمة المتابعة 5. 8.5.

رقم	السيناريو	المتوقع
5.1	حفظ بحث	يظهر في البث المحفوظ
5.2	عدد النتائج الجديدة	العداد يوضح النتائج الجديدة من آخر زيارة
5.3	قائمة متابعة فاضية	حالة فاضية بزرار تصفح

جدول 12 3 سيناريوهات — البث والمتابعة.

التصفّح والاكتشاف 6. 8.6.

رقم	السيناريو	المتوقع
6.1	حالة الفلاتر في URL	شريط العنوان يتغير مع كل حاجة
6.2	مشاركة رابط بالفلاتر	نفس النتيجة بالطبط في تبويب جديد
6.3	زرار الرجوع	الفلاتر السابق يرجع
6.4	ذاكرة التمرير	يرجع لنفس مكان التمرير مش لأول الصفحة
6.5	الانتقال المتحرك	الصورة تتحول بسلاسة للهيرو
6.6	العدادات جنب الفلاتر	كل خيار جنبه رقم، يتغير مع الفلتر
6.7	بحث بكلمة قصيرة	نتائج فيها الكلمة فعليًا، مش تطابق جزئي عشوائي
6.8	شوف المزيد	يضيف نتائج تحت الموجودة، مش يعيد التحميل
6.9	مفيش نتائج	حالة واضحة، مش شاشة فاضية بلا تفسير

جدول 13 9 سيناريوهات — أعقد صفحة في المنتج.

صفحة المشروع الواحد 7. 8.7.

رقم	السيناريو	المتوقع
7.1	الهيرو بالصورة	صورة كاملة العرض، العنوان فوقها
7.2	التبويبات	كل تبويب يعرض المحتوى الصح من غير إعادة تحميل
7.3	تحميل مستند	يبدأ التحميل أو يترفض برسالة واضحة
7.4	اقتراحات بعد القراءة	اقتراحات لمشاريع ثانية ذات صلة

جدول 14 4 سيناريوهات — صفحة المشروع.

غرفة الصفقة 8. 8.8.

رقم	السيناريو	المتوقع
8.1	فتح غرفة الصفقة	كل الأقسام تظهر (الالتزام، الجدول، الشروط، الأسئلة، المستندات، المحادثة)
8.2	إرسال سؤال	يظهر فورًا في القسم المناسب
8.3	الردّ على شروط	الحالة تتحدّث بوضوح

جدول 15 3 سيناريوهات — غرفة الصفقة.

الرسائل والإشعارات 8.9. 9.

رقم	السيناريو	المتوقع
9.1	إرسال رسالة	تظهر فورًا بفقاعة بلون المرسل
9.2	إرفاق ملف	يترفع ويظهر في المحادثة
9.3	تبويبات الإشعارات	«يحتاج قرارك» يعرض المطلوب فعل بس
9.4	حالة فاضية للإشعارات	حالة موجهة، مش شاشة بيضاء

جدول 16 4 سيناريوهات — الرسائل والإشعارات.

الإعدادات والملف الشخصي 8.10. 10.

رقم	السيناريو	المتوقع
10.1	تعديل البروفايل	التغيير يظهر فورًا في أي مكان ثاني
10.2	رفع صورة غلاف/بروفايل	تحدث وتتركب صح
10.3	تغيير كلمة السر	يطلب الحالية ويرفضها لو غلط
10.4	إغلاق الحساب	تحذير واضح إنها لا رجعة فيها
10.5	الملف العام لشخص ثاني	نفس البيانات من غير أضرار تعديل
10.6	قسم «تقدر تتأكد منه»	حقائق قابلة للإثبات، مفيش شارة موثق عامة

جدول 17 6 سيناريوهات — الإعدادات.

الأدمن 8.11. 11.

رقم	السيناريو	المتوقع
11.1	لوحة النظرة العامة	عدّادات وتصوّرات بألوان الهوية
11.2	طابور المراجعة	يقدر يوافق أو يرفض بسبب
11.3	رفض مشروع بسبب	المؤسّس يشوف السبب، يرجع للطابور بعد التعديل
11.4	إدارة المستخدمين	بحث وفلتر شغاليين
11.5	أمان المنصة	قائمة أحداث أمنية حقيقية
11.6	وصول من غير أدمن	يترفض تمامًا برابط مباشر

جدول 18 6 سيناريوهات — الأدمن.

الدفع 8.12. 12.

رقم	السيناريو	المتوقع
12.1	شارة الصندوق التجريبي	واضحة وكبيرة، مش مخفية
12.2	دفع ناجح	تأكيد بأيقونة صح، تنويه Sandbox واضح
12.3	دفع فاشل	رسالة فشل واضحة، الطلب يفضل مفتوح للمحاولة
12.4	دفع طلب مدفوع بالفعل	رفض واضح، مش دفع مزدوج

جدول 19 4 سيناريوهات — الدفع.

الصفحات الثابتة وصفحات الحالة 8.13. 13.

رقم	السيناريو	المتوقع
13.1	الصفحات التعريفية	محتوى يظهر صح باللغتين
13.2	صفحة رفض الوصول	رسالة محترمة، مش خطأ خام
13.3	صفحة الصيانة	رسالة واضحة، مش شاشة بيضاء

جدول 20 3 سيناريوهات — الصفحات الثابتة.

8.14. 14. (RTL) العربي والاتجاه

رقم	السيناريو	المتوقع
14.1	تبدیل اللغة	التخطيط ينقلب بالكامل
14.2	الفلاتر والكروت	انتقلت للناحية الصح بعد التبديل
14.3	نص إنجليزي فاضل	مفیش أي كلمة إنجليزية ظاهرة
14.4	الخطوط تتبدّل	الخط العربي يظهر مش Latin fallback
14.5	مسافات ومحاذاة	مفیش مسافة شمال ثابتة فضلت شمال في العربي

جدول 21 5 سيناريوهات — RTL.

8.15. 15. (Responsive) الاستجابة

رقم	السيناريو	المتوقع
15.1	عرض الموبايل	إعادة ترتيب لعمود واحد، مفیش قصّ أو تراكب
15.2	القائمة على الموبايل	تفتح وتقفّل صح
15.3	النماذج على الموبايل	كل الحقول واضحة من غير تكبير يدوي

جدول 22 3 سيناريوهات — الاستجابة.

خلاصة

15 قسم، 65 سيناريو، النتيجة النهائية: All Pass. لو الدكتور سأل عن أي صفحة، هتلاقها في جدول من التلاتاشر فوق — مفیش صفحة في المنتج من غيرها.

FRONTEND-MANUAL-TEST-PLAN .md

فین ده في الكود
الملف الكامل بنفس الأقسام دي

9. Test Coverage

9.1. Tests Count Is Not Coverage Percentage

اعرف ده كويس

«76 اختبار ناجح» و«Code Coverage» **حاجتين مختلفتين**. عدد الاختبارات بيقول كام سيناريو اتفحص؛ الـ coverage بيقول كام **سطر كود** اتنفذ فعلياً وقت الاختبارات — نسبة مختلفة تماماً ومحتاجة أداة قياس مخصوصة (زي `coverlet` أو `dotnet-coverage`).

خد بالك

لم يتم قياس Code Coverage بنسبة مئوية في المشروع ده. التغطية الحالية مبنية على السيناريوهات والـ endpoints اللي اتفحصت فعلياً — المصادقة، المشاريع، الدفع، الأمان — مش على قياس آلي بنسبة مئوية شاملة للكود كله.

لو الدكتور سأل «إيه نسبة الـ coverage؟» الإجابة الأمينة: «مفستش نسبة مئوية بأداة، بس بغطّي كل الـ endpoints الأساسية في المصادقة والمشاريع والدفع والأمان بسيناريوهات حقيقية.»

10. Regression Testing

اعرف ده كويس

الـ 76 اختبار دول **مش تشغيل واحد وخلاص** — أي تعديل جديد في الباكند ممكن يتشغل عليه `dotnet test` ثاني فوراً، ولو كسر حاجة كانت شغالة قبل كده، الاختبار المرتبط بيها هيفشل فوراً بدل ما الغلط يوصل للمناقشة أو للإنتاج. ده اسمه **Regression Testing** — إعادة تشغيل نفس الاختبارات بعد أي تغيير للتأكد إن القديم لسه شغال.

11. Testing Limitations / Out of Scope

مش كل حاجة في مشروع بالحجم ده تتغطّي، وده أمانة تقوله بوضوح — مش عيب.

الحدّ	التفصيل
مفيش Playwright / E2E آلي للفرونٲ	النطاق اتفق عليه: تكامل آلي للباكند + اختبار يدوي منظم للفرونٲ. أتمتة متصحّ كاملة كانت هتحتاج بنية تحتية جديدة من الصفر لحجم مشروع تخرّج.
الدفع بيتفحص بمزوّد وهمي مش Stripe حقيقي	<code>SimulatedPaymentProvider</code> هو الافتراضي في المشروع أصلاً — بيمشي على نفس آلة الحالة بتاعة أي مزوّد حقيقي، بس من غير اتصال إنترنت فعلي.
مفيش قياس Code Coverage بنسبة مئوية	التغطية الحالية مبنية على الـ endpoints والسيناريوهات اللي اتفحصت، مش على أداة قياس آلي.

الحدّ	التفصيل
الاختبارات الآلية بتغطّي أهم الحدود، مش كل endpoint	المصادقة والمشاريع والدفع والأمان اتغطّوا بعمق؛ endpoints تانية (زي التفاعل والمتابعة) بتغطّيها الـ 48 اختبار الأصلية أو الاختبار اليدوي بس.

جدول 23 حدود نطاق الاختبار الحالي — مش مشاكل، حدود نطاق واعية.

12. Terms You Must Know

المصطلحات اللي ممكن تتسأل فيها مباشرة.

المصطلح	المعنى
Unit test	اختبار دالة واحدة لوحدها، من غير قاعدة بيانات ولا سيرفر.
Integration test	اختبار بيشغّل النظام الحقيقي (سيرفر + قاعدة بيانات) ويتأكد إن الطبقات شغّالة مع بعض.
<code>WebApplicationFactory</code>	أداة بتشغّل APL كامل في الذاكرة، من غير بورت حقيقي.
EF Core InMemory provider	قاعدة بيانات وهمية بتنقذ LINQ على الكائنات مباشرة، مش بترجمه لـ SQL — عكس SQLite أو SQL Server.
Mock / Simulated provider	تنفيذ وهمي لخدمة خارجية (زي <code>SimulatedPaymentProvider</code>) بيتصرّف زي الحقيقي من غير ما يكلم الإنترنت.
<code>[Authorize(Roles = "...")]</code>	فحص الدور — بيتأكد إنك مؤسس أو مستثمر، مش إن الحاجة دي بتاعتك.
Ownership check	فحص إضافي (<code>OwnerId != currentUserId</code>) بيتأكد إن الحاجة فعلاً بتاعتك، مش بس إن دورك يسمح.
User enumeration	تسريب معلومة إن إيميل معيّن مسجّل في المنصة ولا لأ.
Token rotation	كل استعمال لـ refresh token بيولّد واحد جديد ويلغي القديم.
Reuse detection	استخدام توكن مُلغى = إشارة سرقة ⇒ إلغاء كل الجلسات.
Idempotent	نداء العملية مرتين بيدي نفس النتيجة بالطبط.
Manual testing	إنسان بيفتح المنتج بإيده ويجرب، من غير أداة آلية.
Regression	كسر حاجة كانت شغّالة قبل كده بسبب تعديل جديد.

جدول 24 ثلاثة عشر مصطلح.

13. Doctor Questions and Pressure Follow-ups

كل سؤال ومعاه أسئلة المتابعة اللي ممكن تيجي لو ضغط.

مثال

«إيه دورك في التستينج؟»

الإجابة: «بنيت 28 اختبار تكامل بتنده الباكند الحقيقي — مصادقة، مشاريع، دفع، وأمان — فوق الـ48 اختبار الأصلية التي كانت على المنطق الرياضي بس. وثقت 13 محاولة اختراق حقيقية، ونقذت خطة اختبار يدوي بـ65 سيناريو على الفرون كلة.»

لو ضغط: «ليه الـ48 مش كفاية؟» — «لأنها بتختبر دالة لوحدها، مش الـAPI كله شغال مع بعض — مفيش فيها authorization ولا HTTP حقيقي.»

مثال

«إيه الفرق بين Unit وIntegration test ولية محتاجين الاثنين؟»

الإجابة: «الـUnit بيختبر دالة واحدة معزولة، زي `FundingMath`. الـIntegration بيشتغل الـAPI كامل ويبعتله طلب HTTP حقيقي، ويتأكد إن الـrouting والـauthorization وقاعدة البيانات شغالين مع بعض.»

لو ضغط: «وليه محتاجين الاثنين؟» — «الـUnit سريع ودقيق بس محدود؛ الـIntegration أبداً بس بيثبت النظام كله، مش جزء منه. كل واحد بيغطي حاجة التاني مبيقدرش يشوفها.»

مثال

«76 اختبار ناجح معناها 100% Code Coverage؟»

الإجابة: «لا — عدد الاختبارات وCode Coverage حاجتين مختلفتين. العدد بيقول كام سيناريو اتفحص؛ الـcoverage بيقول كام سطر كود اتنفذ فعلياً، ومحتاج أداة قياس مخصوصة. مقستش نسبة مئوية بأداة، بس بغطي كل الـendpoints الأساسية بسيناريوهات حقيقية.»

مثال

«إزاي ضمنت الـTest Isolation؟»

الإجابة: «كل كلاس اختبار بيشتغل على قاعدة بيانات InMemory معزولة خاصة بيه، بتتعمل من جديد كل مرة — مفيش اختبار بيلمس الداتابيز الحقيقية المشتركة ولا بياثر على بيانات فريق تاني.»

مثال

«إزاي بتجهز الـTest Data؟»

الإجابة: «عن طريق `TestHelpers.cs` — بيسجل حساب حقيقي عن طريق الـAPI نفسه، أو يبذر مشروع/ استثمار مباشرة في الداتابيز للحالات التي محتاجة حالة جاهزة من غير ما تمر بكل خطوة يدوياً. كل اختبار بيجهر بياناته بنفسه بإيميل عشوائي، مش بيعتمد على اختبار سبقه.»

مثال

«إيه الـBoundary/Edge Cases اللي اتعملت؟»

الإجابة: «تسجيل بإيميل مكرر، كلمة سر ضعيفة، دخول بكلمة سر غلط 5 مرات متتالية (القفل)، إعادة استخدام refresh token بعد ما يتدور، دفع طلب اتدفع قبل كده، ومحاولة وصول من غير أي توكن أو بتوكن متلاعَب فيه.»

مثال

«إيه الفرق بين Authorization وOwnership؟»

الإجابة: «الـAuthorization بيتأكد من دورك — مؤسس ولا مستثمر. الـOwnership بيتأكد إن الحاجة دي بتاعتك إنت بالذات. مؤسس تاني دوره Innovator برضه، بس اختاري `UpdateProject_ByNonOwnerInnovator_IsForbidden` بيثبت إنه لسه بيترفض لو حاول يعدل مشروع مش بتاعه.»

مثال

«إزاي تختبر Security مش بس بتشوف HTTP Status؟»
 الإجابة: «كل اختبار أمان بيتأكد من الـ status code وكمات من حالة الداتا بيز الفعلية بعد المحاولة — زي اختبار سرقة التوكن اللي بيقرأ كل الـ refresh tokens بعد المحاولة ويتأكد إنهم اتلغوا فعليًا، مش بس إن الرد كان 401.»

مثال

«إشرحلي أقوى اختبار عندك — إيه اللي بيحصل لو استخدم Refresh Token قديم؟»
 الإجابة: «كشف إعادة استخدام الـ refresh token. الاختبار بيدور توكن، وبعدين يستخدم القديم تاني — وده سيناريو سرقة. النظام بيرفض المحاولة وكمات بيلغي كل جلسات الحساب النشطة، مش بس المحاولة اللي فشلت.»
 لو ضغط: «إزاي أناكدت من كده بالكود؟» — «قريت كل الـ refresh tokens بتاعة المستخدم من الداتا بيز بعد المحاولة، وتأكدت إن كلهم عندهم `RevokedAtUtc` مش `null`.»

مثال

«إزاي بتمنع Double Payment؟»
 الإجابة: «طلب تمويل لما يوصل لحالة Paid، أي محاولة checkout جديدة عليه بترفض بـ 400 — اختبار `Checkout_OnAlreadyPaidRequest_IsRejected` بيثبت ده بعد ما يدفع الطلب فعليًا ويحاول يفتح checkout تاني عليه.»

مثال

«رحلة الدفع اتختبرت إزاي من غير Stripe حقيقي؟»
 الإجابة: «فيه مزود وهمي (`SimulatedPaymentProvider`) هو الافتراضي في المشروع أصلًا — بيمشي على نفس آلة الحالة بتاعة أي مزود حقيقي، بس من غير ما يكلم الإنترنت. اختبرت الرحلة كاملة: طلب تمويل، فتح checkout، محاكاة الدفع، وتأكد النتيجة.»
 لو ضغط: «وايه اللي بيضمن إن الفلوس متتسجلش قبل التأكد؟» — «قاعدة `AGENTS.md`: الـ `PaymentEvent` لازم يتسجل قبل أي أثر. اختبرت إن الـ transaction بتتحول لـ `Succeeded` بس بعد خطوة الـ `verify`، مش قبلها.»

مثال

«ليه مغيث Playwright أو E2E Automation؟»
 الإجابة: «اتقيّم النطاق بوعي: أتمتة متصفح كاملة لـ 65 سيناريو مجهود ضخم، ومحتاجة بنية تحتية جديدة من الصفر. القرار كان إن اختبار التكامل الآلي على الباكند + اختبار يدوي منظم وموثق على الفرونت كافيين لحجم مشروع التخرج ده.»

مثال

«إيه حدود الـ Testing الحالي؟»
 الإجابة: «مغيث Playwright/E2E آلي للفرونت — الاعتماد على اختبار يدوي منظم. الدفع بمزود وهمي مش Stripe حقيقي. مغيث قياس `Code Coverage` بنسبة مئوية. والاختبارات الآلية بتغطي أهم الحدود — مصادقة، مشاريع، دفع، أمان — مش كل endpoint في المشروع.»

14. Suggested Study Plan

الجلسة	تقرا	تعمل على الجهاز
الأولى	الفصل 1 و 2 — دورك، أنواع الاختبار، والـ Test Pyramid	شغّل <code>dotnet test</code> وشوف الـ 76 اختبار كلهم ينجحوا.
الثانية	الفصل 3 — البنية والـ Isolation والـ Test Data	افتح <code>CustomWebApplicationFactory.cs</code> و <code>TestHelpers.cs</code> مع بعض.
الثالثة	الفصل 4 و 5 — المصادقة والمشاريع	شغّل <code>dotnet test --filter FullyQualifiedName~AuthEndpointsTests</code> لوحده.
الرابعة	الفصل 6 و 7 — الدفع والأمان	اتبع رحلة الدفع سطر سطر، وافتح <code>SECURITY-FINDINGS.md</code> واربط كل بند باختباره.
الخامسة	الفصل 8 — الاختبار اليدوي	نقّذ 5 سيناريوهات عشوائية بنفسك على الفرونـت الشغّل.
السادسة	الفصل 9 و 10 و 11 — Coverage والـ Regression والحدود	احفظ الفرق بين عدد الاختبارات والـ coverage، وجّهز إجابة «إيه حدود شغلّك».
الأخيرة	الفصل 12 و 13 — المصطلحات وأسئلة الضغط	جاوب على كل مثال بصوت عالي من غير ما تبصّ، وبعدها الـ Cheat Sheet.

جدول 25 سبع جلسات، كل واحدة قراءة + حاجة بإيدك.

اعرف ده كويس

أهم نصيحة: **متذاكرش بالقراءة بس**. شغّل `dotnet test` بنفسك وشوف الـ 76 اختبار ينجحوا قدامك. الدكتور مش بيسأل «إيه تعريف الـ Integration Test» — بيسأل «ورّيني» و«ليه».

15. Final Code Map — Where to Find Everything

مرجع واحد يجمع كل حاجة اتشرحت في الملف. لو الدكتور سأل «فين الكود بتاع الحاجة دي؟» — افتح الصفحة دي بس.

What	Where in Code	What it does
Test server setup	<code>Integration/CustomWebApplicationFactory.cs</code>	يشغّل الـ API كامل في الذاكرة على قاعدة بيانات InMemory معزولة.
Test helpers	<code>Integration/TestHelpers.cs</code>	تسجيل، دخول، بذر مشاريع واستثمارات مباشرة في الداتابيز.
Log capture	<code>Integration/TestLogCapture.cs</code>	بتلقط الأخطاء اللي الـ exception handler بيخفيها عن رد الـ HTTP.

What it does	Where in Code	What
تسجيل، دخول، تدوير التوكن وكشف السرقة، القفل، تأكيد الإيميل.	Integration/ AuthEndpointsTests.cs	Auth tests
إنشاء مشروع، العدّادات، إقفال جولة، الملكية.	Integration/ ProjectsEndpointsTests.cs	Projects tests
رحلة الدفع الكاملة، ومنع الدفع المزدوج.	Integration/ PaymentsEndpointsTests.cs	Payments tests
سبع محاولات اختراق مباشرة.	Integration/ SecurityBoundaryTests.cs	Security tests
توثيق كامل للـ 13 محاولة اختراق ونتيجة كل واحدة.	SECURITY-FINDINGS.md	Security findings
65 سيناريو على كل صفحة في الفرونت، النتيجة All Pass.	FRONTEND-MANUAL-TEST-PLAN.md	Manual test plan
الـ 48 اختبار الأصلية — مش بتاعتك، بس السويت بتاعتك بتبني فوقها.	FundingMathTests.cs PipelineStagesTests.cs	Original domain tests

جدول 26 كل حاجة في نصيبك، في جدول واحد.

16. Cheat Sheet — Review Before You Walk In

خلاصة

نصيبك: 28 اختبار تكامل (مصادقة، مشاريع، دفع، أمان) + توثيق 13 محاولة اختراق + خطة اختبار يدوي بـ 65 سيناريو.
76 اختبار آلي، صفر فشل.

مش نصيبك: الـ 48 اختبار الأصلية، كود الباكند والفرونت نفسه، Playwright (اتقيّم واتلغى بوعي).

أنواع الاختبار (Test Pyramid): Unit (دالة لوحدها) → Integration (النظام كله مع بعض) → Manual/E2E (عين بشرية) — كل طبقة بتغطّي حاجة الثانية مبتقدرش تشوفها.

العزل: كل كلاس اختبار على قاعدة بيانات InMemory معزولة، بتعمل من جديد كل مرة. `TestHelpers.cs` بيجهّز الحسابات والبيانات — كل اختبار بيجهّز بياناته بنفسه، مش بيعتمد على اختبار تاني.

أقوى اختبار: كشف إعادة استخدام الـ refresh token — توكن مُلغى بيتستخدم = سرقة ⇒ إلغاء كل الجلسات، مش بس المحاولة.

رحلة الدفع: طلب ← checkout ← محاكاة الدفع ← verify (دي اللي بتطبّق النتيجة فعليًا). كله على مزوّد وهمي، مفيش Stripe حقيقي.

الأمان: 13 محاولة اختراق حقيقية، **كلها فشلت** — كل اختبار بيتأكد من حالة الداتابيز بعد المحاولة، مش بس الـ status code.

الاختبار اليدوي: 65 سيناريو، 15 قسم، **All Pass**.

Coverage: عدد الاختبارات ≠ نسبة تغطية الكود. مفيش قياس بأداة — التغطية مبنية على الـ endpoints والسيناريوهات الفعلية.

Regression: نفس الـ 76 اختبار تتشغّل تاني بعد أي تعديل عشان تتأكد إن القديم لسه شغّال.

حدود النطاق: مفيش Playwright/E2E آلي، الدفع بمزوّد وهمي، مفيش قياس Coverage بنسبة — قرارات نطاق واعية، مش حاجات فانت.